UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

SANTO DOMINGO



Estudiante

Casas Castaño Michael Alejandro

Tema

Examen U1

Materia

Aplicaiones Distribuidas

Docente

Mg. Geovanny Brito

Fecha

04/11/2026

SANTO DOMINGO – ECUADOR

Introducción

Los sistemas distribuidos se definen como un conjunto de componentes independientes que se presentan al usuario como un sistema único y coherente (Tanenbaum & Van Steen, 2017). En este tipo de sistemas, varios clientes consultan y modifican datos compartidos al mismo tiempo, por lo que mantener la consistencia de la información y un buen tiempo de respuesta es un desafío central (Coulouris et al., 2012). Una aplicación CRUD tradicional basada en solicitudes HTTP no resuelve bien este problema: cada cliente solo conoce el estado de los datos en el momento de su última consulta, y puede trabajar con información desactualizada si otro usuario la modificó entre tanto.

Para reducir esta limitación, el protocolo WebSocket habilita un canal de comunicación bidireccional y persistente entre cliente y servidor, de modo que el servidor puede enviar información sin esperar una nueva solicitud (Fette & Melnikov, 2011). En este trabajo se usa Socket.IO, una librería que implementa esta comunicación sobre Node.js, para notificar a todos los clientes conectados cada vez que se crea, edita o elimina un registro de la tabla `content` de la base de datos `primevideodb`. Así, las pantallas de distintos usuarios se mantienen sincronizadas sin recargar la página.

Por otro lado, Node.js ejecuta el código JavaScript en un único hilo con un modelo de eventos no bloqueante, lo que funciona bien para tareas de entrada y salida, pero se degrada cuando se ejecutan cálculos intensivos que ocupan la CPU (Tilkov & Vinoski, 2010). Para evitarlo, el módulo `worker_threads` permite ejecutar código JavaScript en hilos paralelos, de modo que el procesamiento pesado no detenga la atención de las demás peticiones (OpenJS Foundation, n.d.). En esta práctica se emplea un Worker para generar estadísticas sobre los más de 9 600 contenidos almacenados, y se compara el comportamiento de la versión inicial del CRUD con el de la versión mejorada con WebSockets y concurrencia.

Objetivos

Objetivo general

Desarrollar una aplicación CRUD sobre la tabla `content` de la base de datos `primevideodb`, incorporando comunicación en tiempo real mediante WebSockets y procesamiento concurrente mediante hilos, para analizar el comportamiento de la aplicación antes y después de estas mejoras.

Objetivos específicos

Verificar que la base de datos `primevideodb` esté correctamente cargada en SQL Server, comprobando el total de registros y visualizando una muestra de ellos.

Implementar las operaciones de listar, registrar, editar y eliminar contenidos, conectadas a `primevideodb`.

Incorporar Socket.IO para que los cambios realizados por un cliente se reflejen automáticamente en otros clientes conectados, sin recargar la página.

Implementar un procesamiento estadístico sobre los contenidos usando un Worker Thread, sin bloquear el hilo principal de la aplicación.

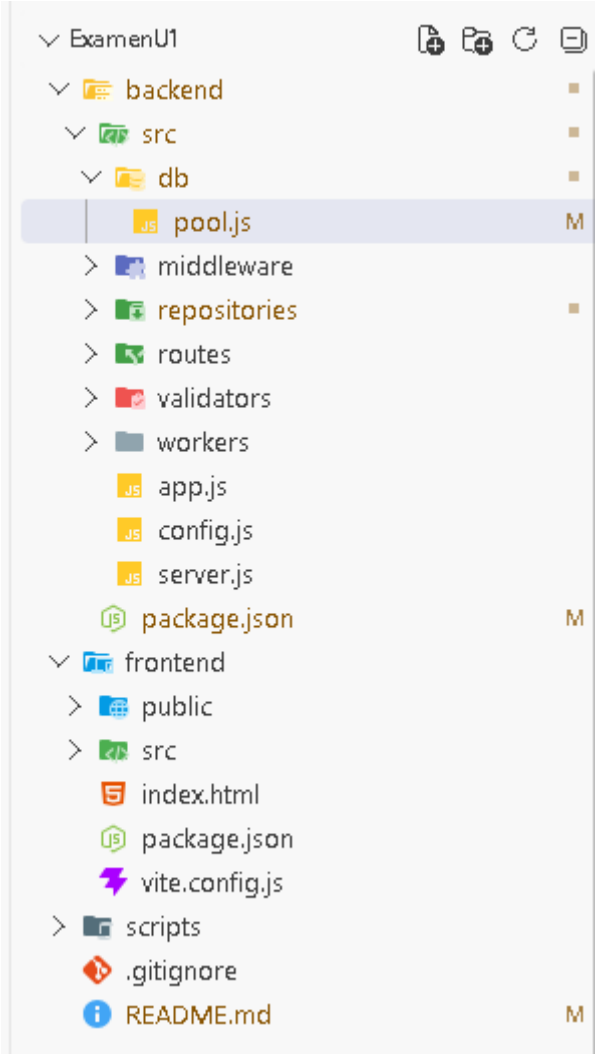
Comparar la versión inicial del CRUD con la versión mejorada, identificando las ventajas de la comunicación en tiempo real y la concurrencia.

Desarrollo

1. Verificación inicial de la base de datos

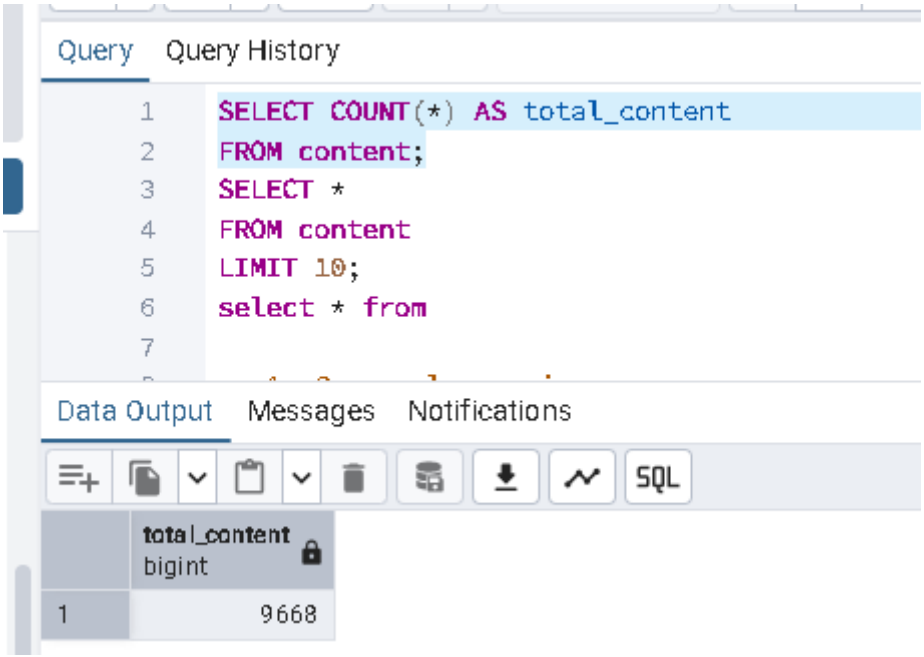
Figura 1

Estructura del proyecto



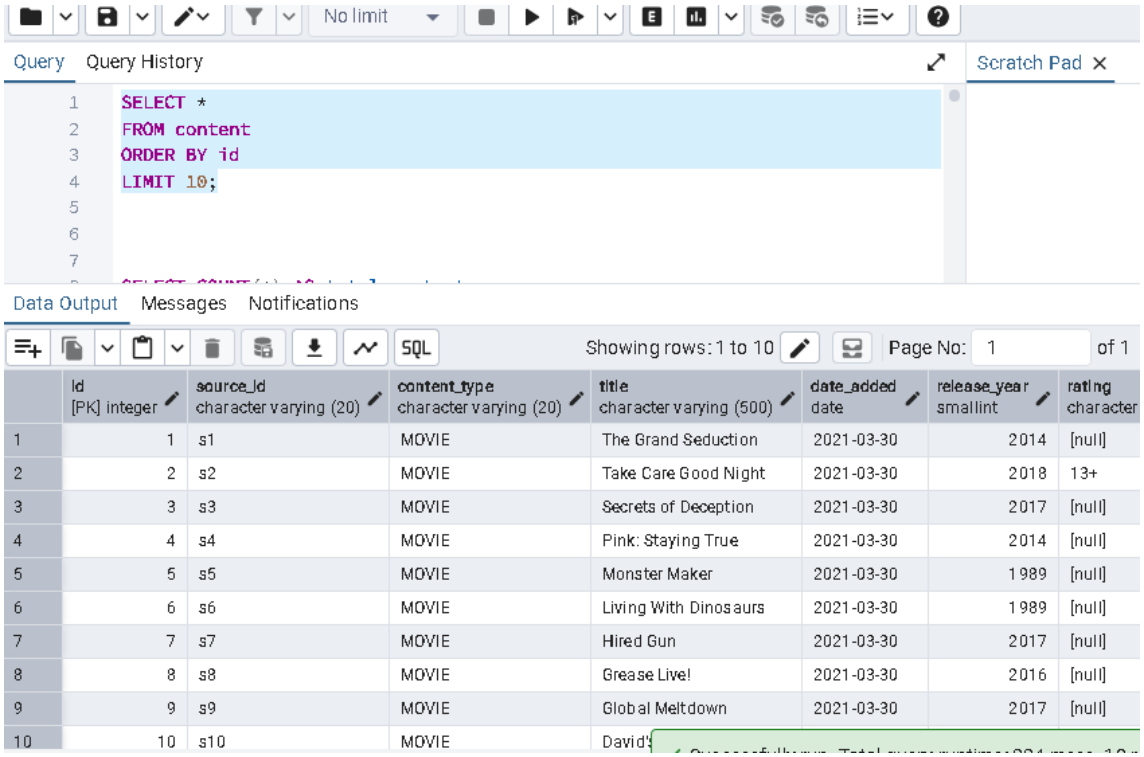
El proyecto utiliza las tecnologías de Express, y react para back y front respectivamente, además se utiliza la versión para la base de datos de postgres sql.

Figura 2
Verificación del total de registros en la tabla



En esta captura se observa SQL Server Management Studio conectado a localhost,1433, con la base de datos primevideodb seleccionada. Se ejecutó la consulta SELECT COUNT(*) AS total_content FROM content;, que devolvió [9668] registros. Esto confirma que el script PrimeVideoDB_SQLServer.sql se cargó completo antes de empezar el desarrollo.

Figura 2
Primeros 10 registros de la tabla content

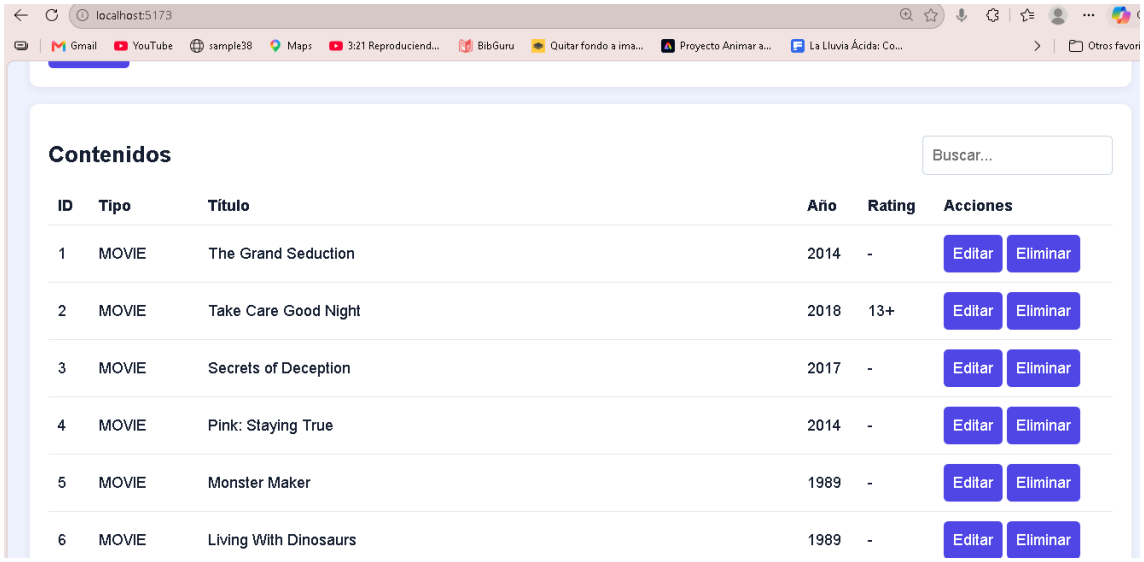


	id [PK] integer	source_id character varying (20)	content_type character varying (20)	title character varying (500)	date_added date	release_year smallint	rating character
1	1	s1	MOVIE	The Grand Seduction	2021-03-30	2014	[null]
2	2	s2	MOVIE	Take Care Good Night	2021-03-30	2018	13+
3	3	s3	MOVIE	Secrets of Deception	2021-03-30	2017	[null]
4	4	s4	MOVIE	Pink: Staying True	2021-03-30	2014	[null]
5	5	s5	MOVIE	Monster Maker	2021-03-30	1989	[null]
6	6	s6	MOVIE	Living With Dinosaurs	2021-03-30	1989	[null]
7	7	s7	MOVIE	Hired Gun	2021-03-30	2017	[null]
8	8	s8	MOVIE	Grease Live!	2021-03-30	2016	[null]
9	9	s9	MOVIE	Global Meltdown	2021-03-30	2017	[null]
10	10	s10	MOVIE	David's			

Se ejecutó SELECT TOP 10 * FROM content ORDER BY id; sobre primevideodb. El resultado muestra las columnas [id, title, type, genre, release_year, ...] con datos reales, lo que demuestra que la tabla es consultable y que su estructura corresponde a la esperada.

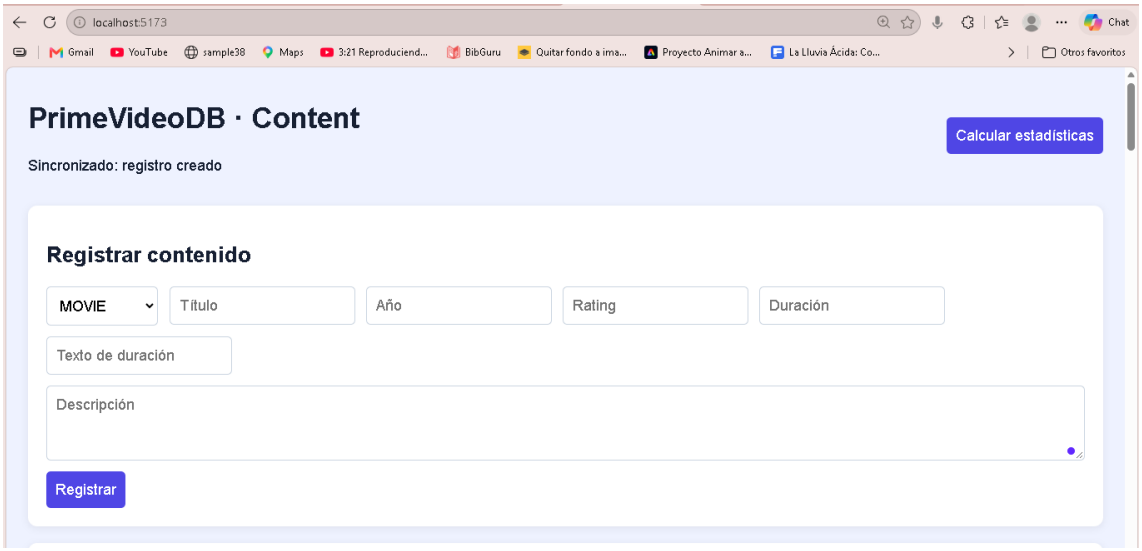
2. CRUD básico

Figura 3
Listado de contenidos (Read)



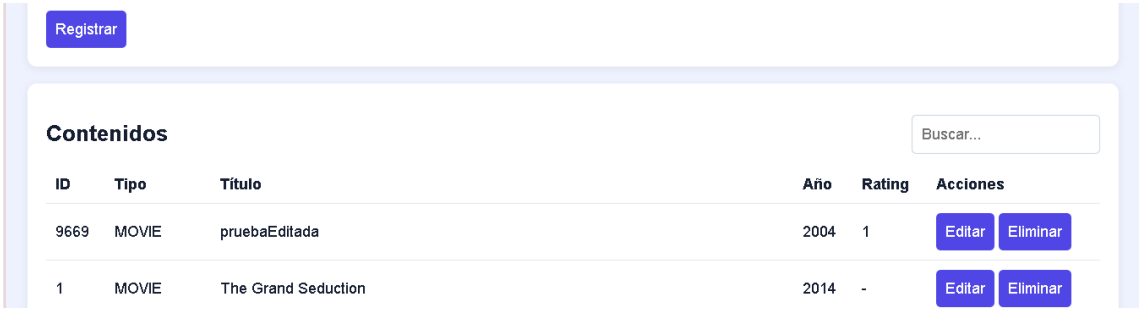
Pantalla principal de la aplicación React, que consume GET /api/content?page=1&limit=20. Muestra los contenidos de primevideodb en una tabla paginada, con [título, tipo, género, año]. La paginación evita traer los 9668 registros en una sola petición.

Figura 4
Registro de un nuevo contenido (Create)



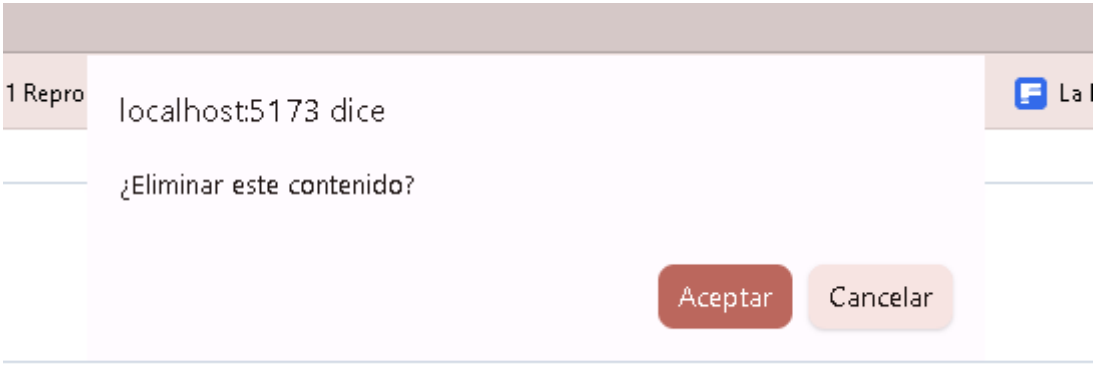
Formulario para crear un contenido, enviado con POST /api/content. Tras guardar, el nuevo registro aparece en la tabla con su id autogenerado. El backend valida los campos obligatorios y responde con código 201 si se creó correctamente.

Figura 5
Edición de un contenido (Update)



Se seleccionó el registro [nombre del contenido] y se modificó [campo modificado] mediante `PUT /api/content/:id`. La tabla refleja el valor actualizado, lo que confirma que el cambio se guardó en la base de datos.

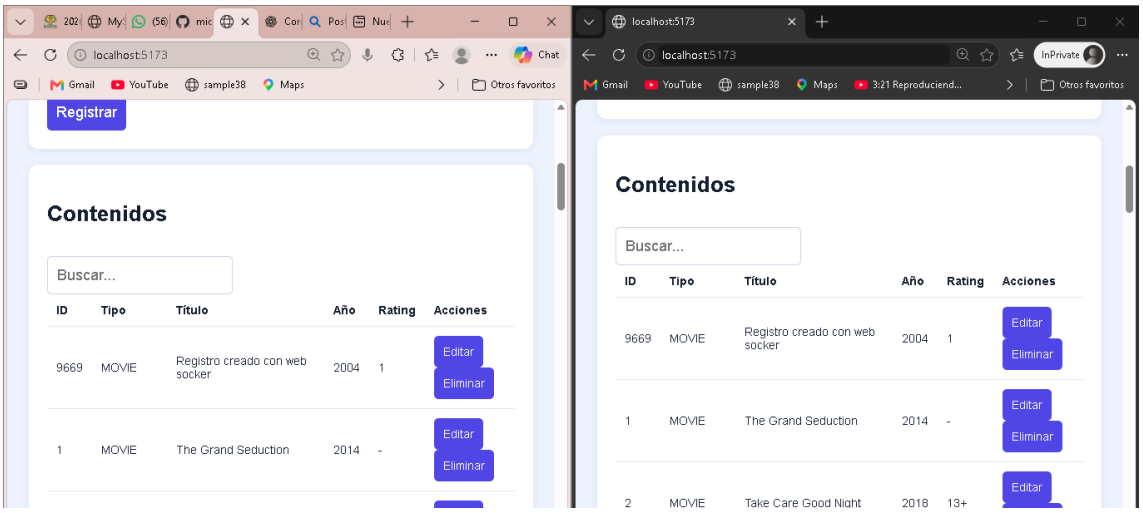
Figura 6
Eliminación de un contenido (Delete)



Se eliminó el registro [nombre] con `DELETE /api/content/:id`. La captura muestra que ya no aparece en la lista. También se verificó con `SELECT * FROM content WHERE id = [id]` en SSMS, que no devuelve filas.

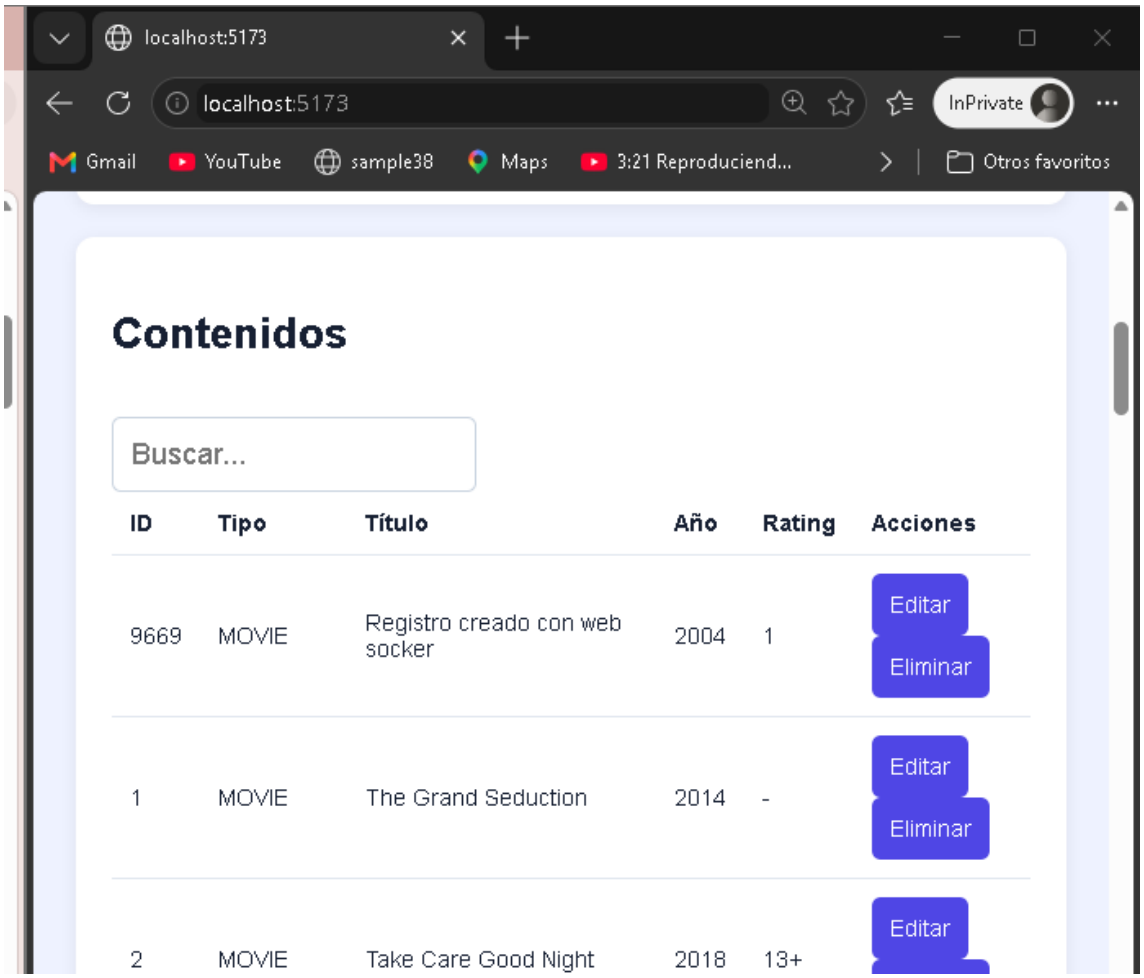
3. Mejora con WebSockets

Figura 7
Cliente A registra un contenido con dos ventanas abiertas



Se abrieron dos clientes lado a lado: un navegador normal y una ventana de incógnito, ambos en http://localhost:[puerto]. En el Cliente A se registró el contenido [nombre]. El servidor, tras guardar en la BD, emite el evento content:created mediante Socket.IO a todos los clientes conectados.

Figura 8
Cliente B se actualiza automáticamente, sin recargar



En la segunda ventana aparece el contenido [nombre] sin haber pulsado F5 ni recargado la página. Lo mismo se comprobó para la edición (`content:updated`) y la eliminación (`content:deleted`). Esto demuestra la comunicación bidireccional en tiempo real entre el servidor y varios clientes.

4. Mejora mediante hilos

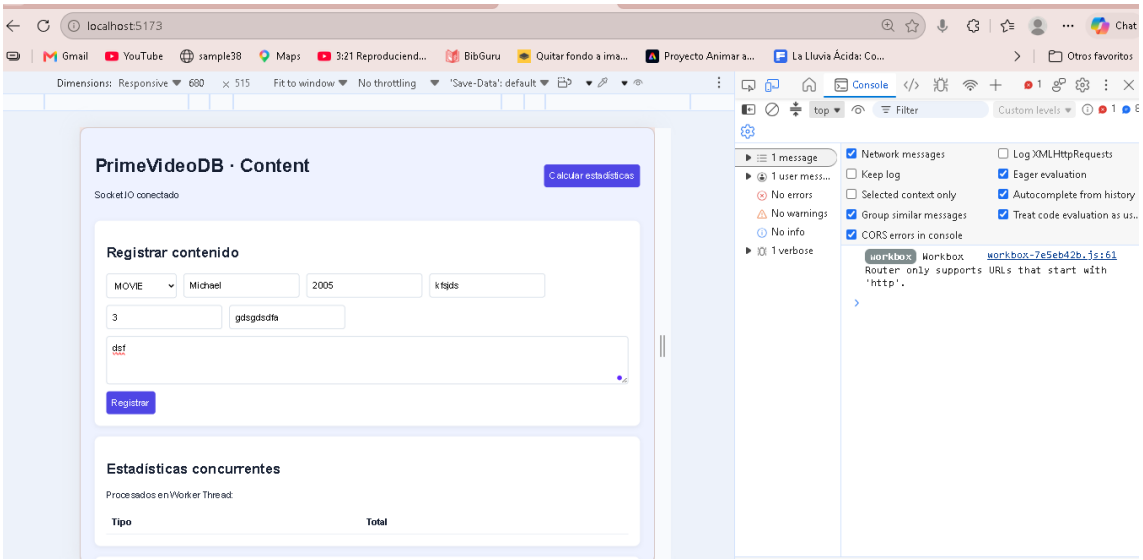
Figura 9
Estadística de contenidos por género procesada en un Worker Thread

```
import { parentPort } from 'node:worker_threads';

parentPort.on('message', (records) => {
  if (!Array.isArray(records)) return;
  const byType = {};
  const byYear = {};
  for (const record of records) {
    byType[record.content_type] = (byType[record.content_type] || 0) + 1;
    if (record.release_year) byYear[record.release_year] = (byYear[record.release
  }
  parentPort.postMessage({
    processed: records.length,
    byType,
    byYear: Object.fromEntries(Object.entries(byYear).sort(([a], [b]) => Number(
  });
});
```

Se implementó el endpoint `GET /api/content/stats`, que lanza un `Worker` de `worker_threads`. El hilo recorre los registros de `content` y calcula [la cantidad de contenidos por género y la proporción de películas frente a series]. El hilo principal recibe el resultado por mensajes (`postMessage`) y lo envía a la interfaz. La captura muestra [una tabla o gráfico] con los resultados y el tiempo de procesamiento [X ms].

Figura 10
El CRUD sigue respondiendo mientras el hilo trabaja



Mientras el Worker procesaba la estadística, se ejecutó una operación del CRUD ([listar o registrar]) y respondió sin esperar. Esto evidencia que el procesamiento pesado no bloquea el event loop del hilo principal. [Opcional: incluir captura de la pestaña Network con ambas peticiones o logs de consola con los tiempos.]

5. Comparación entre la versión inicial y la mejorada

¿Qué cambió en el CRUD al incorporar WebSockets? En la versión inicial, cada cliente solo veía los cambios al recargar la página o volver a consultar la API. Con Socket.IO, el servidor notifica a todos los clientes conectados cada vez que alguien crea, edita o elimina un contenido, y las pantallas se actualizan solas. Las operaciones CRUD siguen usando HTTP; los WebSockets se añaden como canal de notificación, sin reemplazarlas.

¿Qué ventaja se obtuvo al utilizar un hilo? Calcular estadísticas sobre miles de registros es trabajo de CPU. En Node.js, si se hace en el hilo principal, bloquea el event loop y la aplicación deja de responder a otras peticiones mientras tanto. Con `worker_threads`, ese cálculo corre en paralelo en otro hilo, y el CRUD y los WebSockets siguen atendiendo a los usuarios sin retrasos.

¿Qué diferencia se observa entre ambas versiones? La versión inicial era un CRUD aislado: cada usuario veía datos que podían estar desactualizados y cualquier tarea pesada afectaba a todos. La versión mejorada es una aplicación distribuida reactiva: los clientes comparten el estado en tiempo real y el procesamiento intensivo se separa del flujo principal. Esto mejora la experiencia de usuario, la consistencia entre clientes y la escalabilidad de la aplicación.

Enlace al repositorio: michaelCasasC/U1-Examen-AP-CasasMichael

Bibliografía

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). *Distributed systems: Concepts and design* (5.ª ed.). Addison-Wesley.

Fette, I., & Melnikov, A. (2011). *The WebSocket protocol* (RFC 6455). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc6455

OpenJS Foundation. (s. f.). *Worker threads*. Node.js Documentation. https://nodejs.org/api/worker_threads.html

Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed systems* (3.ª ed.). Distributed-Systems.net.

Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. *IEEE Internet Computing, 14*(6), 80-83. https://doi.org/10.1109/MIC.2010.145